Comparative Analysis of Databases

Poojal Katiyar (220770)
Pallav Goyal (220747)
Nipun Nohria (220717)
Ansh Agarwal (220165)

April 26, 2026

Contents

1 Abstract **2**

2 Motivation and Problem Statement **2**

3 Dataset Description and Preprocessing **2**
3.1 Relational Schema 2

4 Setup **3**
4.1 Neo4j 3
4.2 MongoDB 4
4.3 PostgreSQL 4

5 Methodology **4**
5.1 Neo4j (Graph Database) 4
5.1.1 Data Model 4
5.1.2 Relationships 4
5.2 MongoDB (Document Database) 5
5.2.1 Data Model 5
5.2.2 Example Document: Badges Collection 5
5.2.3 Hierarchical Data and Document Nesting 5
5.3 PostgreSQL (Relational Database) 6
5.3.1 Data Model 6
5.3.2 Schema Definition 6

6 Query Comparison Results **6**

7 Inferences **8**

8 Conclusion **9**

9 Challenges **10**

10 Future Scope **10**

11 Code Link **10**

1 Abstract

Modern applications require different data management strategies depending on the problem at hand. This report covers three widely-used database systems: Neo4j, MongoDB, and PostgreSQL. Each follows a different data model and is suited for different types of workloads. The goal is to understand when and why to choose one over the other.

2 Motivation and Problem Statement

For data-driven applications, choosing the right database system becomes a critical design decision. Each database paradigm offers unique advantages. This project is motivated by the need to systematically compare these database systems under a common framework to understand their strengths, limitations, and practical applicability. The objective of this project is to compare different database systems—MySQL (relational), MongoDB (document-based), and Neo4j (graph-based)—using a common dataset. We evaluated their performance on a variety of query types.

3 Dataset Description and Preprocessing

For this project, the **Superuser dataset** from the Stack Exchange data dump was selected. The dataset size is approximately **4.3 GB** and consists of **8 XML files**, including `Posts.xml`, `Users.xml`, `Comments.xml`, `Votes.xml`, `Tags.xml`, `PostLinks.xml`, `Badges.xml`, and `PostHistory.xml`.
The dataset contains information about user accounts on stack exchange. It contains the posts by the users and comments made on those posts.It contains badges awarded to users. It also contains the votes by different users.

For efficient database loading, the dataset was converted into **CSV format** using Python.

3.1 Relational Schema

Users(*Id*, AccountId, Reputation, DisplayName, AboutMe, CreationDate, LastAccessDate, WebsiteUrl, Location, Views, UpVotes, DownVotes)

Tags(*Id*, TagName, Count, WikiPostId, ExcerptPostId)

Posts(*Id*, PostTypeId, OwnerUserId, ParentId, AcceptedAnswerId, CreationDate, LastEditDate, LastActivityDate, ClosedDate, Score, ViewCount, AnswerCount, CommentCount, FavoriteCount, Title, Body, Tags, ContentLicense, LastEditorUserId, LastEditorDisplayName, OwnerDisplayName)

Comments(*Id*, PostId, UserId, CreationDate, Text, Score, UserDisplayName)

Badges(*Id*, UserId, Name, Date, Class, TagBased)

Votes(*Id*, PostId, UserId, VoteTypeId, CreationDate, BountyAmount)

PostLinks(*Id*, PostId, RelatedPostId, LinkTypeId, CreationDate)

PostHistory(*Id*, PostId, UserId, CreationDate, PostHistoryTypeId, RevisionGUID, Text, Comment, ContentLicense, UserDisplayName)

The ER diagram for the dataset is as follows:

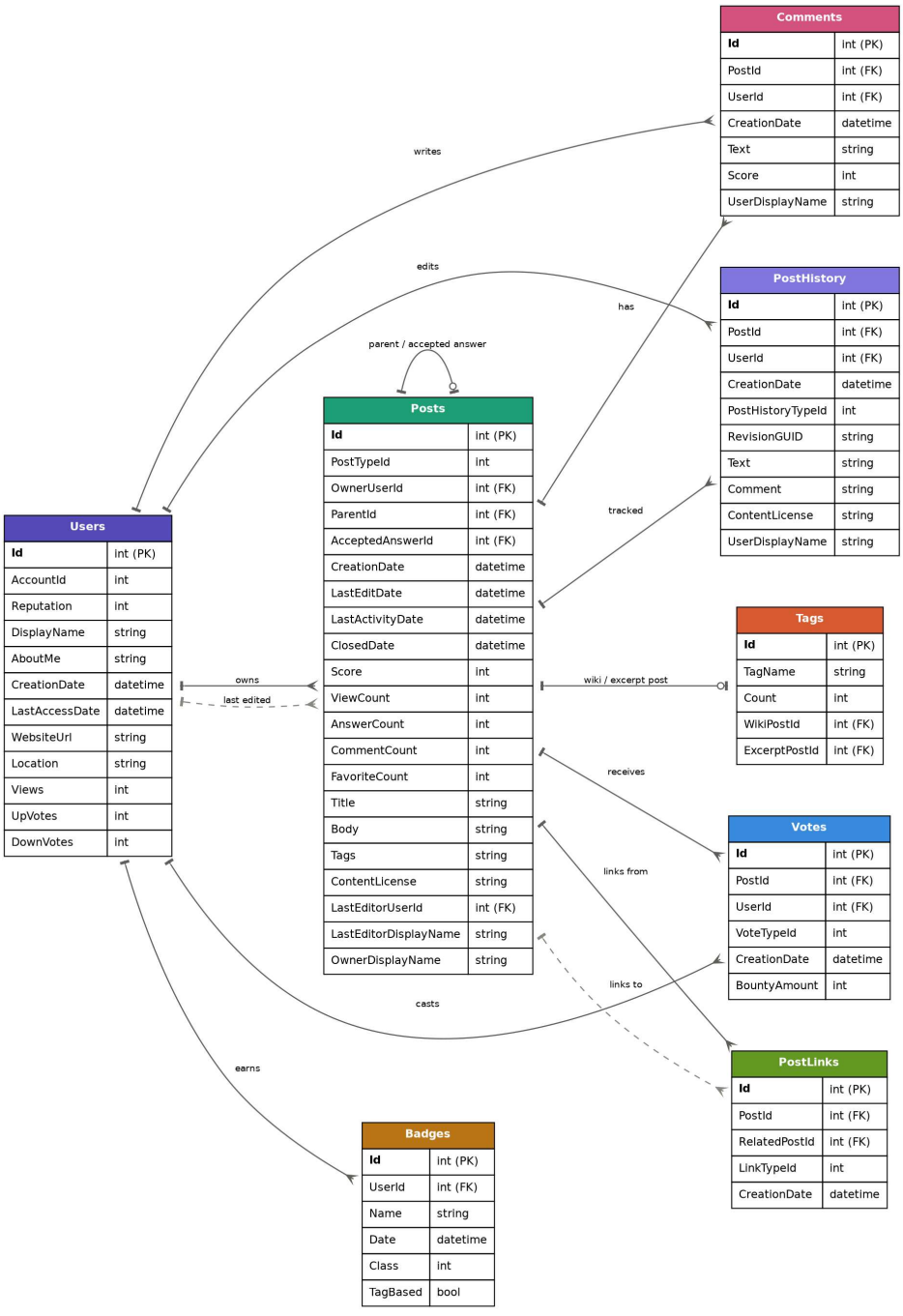


Figure 1: ER Diagram

4 Setup

4.1 Neo4j

1. We installed neo4j using apt on ubuntu.
2. Testing is done using cypher queries and an evaluation script.

4.2 MongoDB

1. **Install Dependencies:** Install the necessary tools for handling GPG keys and downloading files.
2. **Import GPG Key:** Download and dearmor the official MongoDB public GPG key to ensure package authenticity.
3. **Configure Repository:** Create the list file for the MongoDB repository using the system's codename (e.g., focal, jammy).
4. **Install MongoDB:** Update the local package database and install the `mongodb-org` meta-package.

4.3 PostgreSQL

1. Installed via package manager:

```
sudo apt install postgresql   # Ubuntu/Debian
```

5 Methodology

5.1 Neo4j (Graph Database)

5.1.1 Data Model

In the Neo4j implementation, entities are represented as **Nodes** and their interactions as **Relationships**. This allows for traversing deep connections (like post history or user interactions) without the overhead of relational joins.

Node Labels: `Users`, `Posts`, `Tags`, `Badge`, `Comments`, `PostLinks`, `Votes`, `PostHistory`.

5.1.2 Relationships

The following directed relationships define the graph topology. In this model, relationships are treated as first-class citizens, allowing for efficient traversal of the schema:

- (User) $\xrightarrow{\text{AUTHORED}}$ (Post)
- (User) $\xrightarrow{\text{WROTE_COMMENT}}$ (Comment)
- (User) $\xrightarrow{\text{WROTE_HISTORY}}$ (PostHistory)
- (User) $\xrightarrow{\text{EDITED}}$ (Post)
- (User) $\xrightarrow{\text{EARNED}}$ (Badge)
- (User) $\xrightarrow{\text{CAST}}$ (Vote)
- (Post) $\xrightarrow{\text{HAS_TAG}}$ (Tag)
- (Post) $\xrightarrow{\text{ACCEPTED_ANSWER}}$ (Post)
- (Post) $\xrightarrow{\text{REPLY_TO}}$ (Post)
- (Post) $\xrightarrow{\text{LINKS_TO}}$ (Post)
- (Comment) $\xrightarrow{\text{COMMENT_ON}}$ (Post)
- (Comment) $\xrightarrow{\text{ON_POST}}$ (Post)

- (PostHistory) $\xrightarrow{\text{REVISION_OF}}$ (Post)
- (Tag) $\xrightarrow{\text{HAS_WIKI_POST}}$ (Post)
- (Tag) $\xrightarrow{\text{HAS_EXCERPT_POST}}$ (Post)

5.2 MongoDB (Document Database)

5.2.1 Data Model

MongoDB stores data in flexible, JSON-like **BSON documents**. This is ideal for handling the semi-structured nature of posts and comments where fields may vary.

5.2.2 Example Document: Badges Collection

Based on the provided dataset, a document in the `Badges` collection follows this structure:

```
{
  "_id": ObjectId("69ec65ff3d5d388b98dec8c8"),
  "Date": "2009-07-15T07:12:30.673",
  "Name": "Teacher",
  "UserId": 7,
  "TagBased": false,
  "Id": 11,
  "Class": 3
}
```

Listing 1: Simple document structure in the Badges collection

5.2.3 Hierarchical Data and Document Nesting

MongoDB is also capable of representing complex, nested relationships within a single document. This is particularly useful for embedding related entities, such as a user's posts and their subsequent comments, as shown in the following `Users` document example:

```
{
  "_id": ObjectId("69ecfa77010afc9f47a31422"),
  "DisplayName": "Chris Disbro",
  "Reputation": 1294,
  "CreationDate": "2010-04-12T19:29:37.870",
  "LastAccessDate": "2017-03-23T13:10:24.523",
  "MyPosts": [
    {
      "Id": 131032,
      "PostTypeId": 2,
      "Body": "<p>OK I want to make sure I understand this...</p>",
      "CreationDate": "2010-04-14T20:26:15.737",
      "Score": 0,
      "Comments": [
        {
          "Id": 133532,
          "UserId": 34218,
          "Text": "yes, its the oddest thing i have ever seen",
          "CreationDate": "2010-04-14T20:33:07.363",
          "Score": 0
        },
        { "...": "..." }
      ]
    },
    { "...": "..." }
  ]
}
```

Listing 2: Example of a nested document structure

To explore the importance of proper data formatting, we evaluated MongoDB using two different schema designs. In the first approach, the CSV files were directly converted into collections, preserving a flat structure similar to that of a relational (SQL) database.

In the second approach, we restructured the data to leverage MongoDB's document-oriented model. Since the original dataset consisted of flat tables, a Python script was used to introduce hierarchical nesting—embedding Posts within Users and Comments within Posts. This nested schema better aligns with MongoDB's strengths in handling hierarchical and denormalized data. That is also mentioned as comparison in the table below under MongoDb(nested)

5.3 PostgreSQL (Relational Database)

5.3.1 Data Model

PostgreSQL enforces strict data integrity through a normalized relational schema. Below is the definition for the central `Users` entity.

5.3.2 Schema Definition

```
-- Users Table Definition example
CREATE TABLE "Users" (
    "Id" INTEGER PRIMARY KEY,
    "AccountId" INTEGER,
    "Reputation" INTEGER DEFAULT 0,
    "DisplayName" VARCHAR(255),
    "AboutMe" TEXT,
    "CreationDate" TIMESTAMP,
    "LastAccessDate" TIMESTAMP,
    "WebsiteUrl" TEXT,
    "Location" TEXT,
    "Views" INTEGER DEFAULT 0,
    "UpVotes" INTEGER DEFAULT 0,
    "DownVotes" INTEGER DEFAULT 0
);
```

6 Query Comparison Results

We evaluated all database systems using a common dataset and executed a set of 11 queries across each system. Each query was run for 10 iterations to ensure consistent performance measurement. All experiments were conducted on a Linux-based operating system.

It was observed that the first iteration of each query was slower compared to subsequent runs. This is primarily due to caching effects, as the data is loaded into memory during the initial execution, resulting in faster query performance in later iterations.

Table 1: Query Benchmark Comparison Across Database Systems (10 iterations each)

Query Description	**PostgreSQL**	**Neo4j**	**MongoDB**	**MongoDB (Nested)**
List all tags used in more than 1,000 posts (P1)	Avg: 0.94 ms Warm: 0.76 ms	Avg: 25.0 ms Warm: 4.8 ms	Avg: 14.9 ms Warm: 4.5 ms	Avg: 39.4 ms Warm: 34.6 ms
Comments containing the word 'error' (P2)	Avg: 1412.8 ms Warm: 1395.0 ms	Avg: 159.5 ms Warm: 124.1 ms	Avg: 2371.5 ms Warm: 2319.9 ms	Avg: 2202.7 ms Warm: 2134.0 ms
"Power Users" with ≥ 1 post AND ≥ 1 comment (P3)	Avg: 1225.6 ms Warm: 1159.3 ms	Avg: 951.1 ms Warm: 671.2 ms	Avg: 3564.0 ms Warm: 3285.8 ms	Avg: 3997.7 ms Warm: 3886.3 ms
Active users (created post OR left comment) (P4)	Avg: 1515.2 ms Warm: 1491.5 ms	Avg: 67.5 ms Warm: 45.3 ms	Avg: 2976.3 ms Warm: 2802.0 ms	Avg: 3470.4 ms Warm: 3411.0 ms
Top 10 users by reputation in 'New York' (P5)	Avg: 206.0 ms Warm: 175.9 ms	Avg: 1089.6 ms Warm: 875.5 ms	Avg: 782.3 ms Warm: 725.8 ms	Avg: 39.4 ms Warm: 33.0 ms
Comments by 'JohnDoe' ordered by date (P6)	Avg: 119.0 ms Warm: 115.9 ms	Avg: 487.3 ms Warm: 309.7 ms	Avg: 719.4 ms Warm: 624.2 ms	Avg: 42.7 ms Warm: 39.4 ms
Count distinct badge types for a user (P7)	Avg: 88.6 ms Warm: 75.1 ms	Avg: 767.9 ms Warm: 574.1 ms	Avg: 639.7 ms Warm: 602.6 ms	Avg: 35.0 ms Warm: 30.7 ms
Users commenting on own posts > 5 times (P8)	Avg: 1381.8 ms Warm: 1335.5 ms	Avg: 26.5 ms Warm: 24.3 ms	Timeout	Avg: 745.3 ms Warm: 731.0 ms
Answers accepted by elite users (>10K rep) (P9)	Avg: 663.9 ms Warm: 641.3 ms	—	—	Timeout
User with post having most comments (P10)	Avg: 2697.5 ms Warm: 2671.8 ms	Avg: 1728.4 ms Warm: 1655.8 ms	Avg: 2503.3 ms Warm: 2489.0 ms	Avg: 888.7 ms Warm: 876.2 ms
User profile page (top 5 posts + 3 recent comments each) (P11)	Avg: 335.2 ms Warm: 307.2 ms	Avg: 4176.6 ms Warm: 3038.5 ms	Avg: 5956.7 ms Warm: 5124.4 ms	Avg: 44.9 ms Warm: 42.3 ms

Note: "Avg" = average over all 10 runs; "Warm" = average excluding the first (cold) run. All times are wall-clock. "Timeout" indicates the query did not complete within the allowed time. "—" indicates the query was not executed on that database.

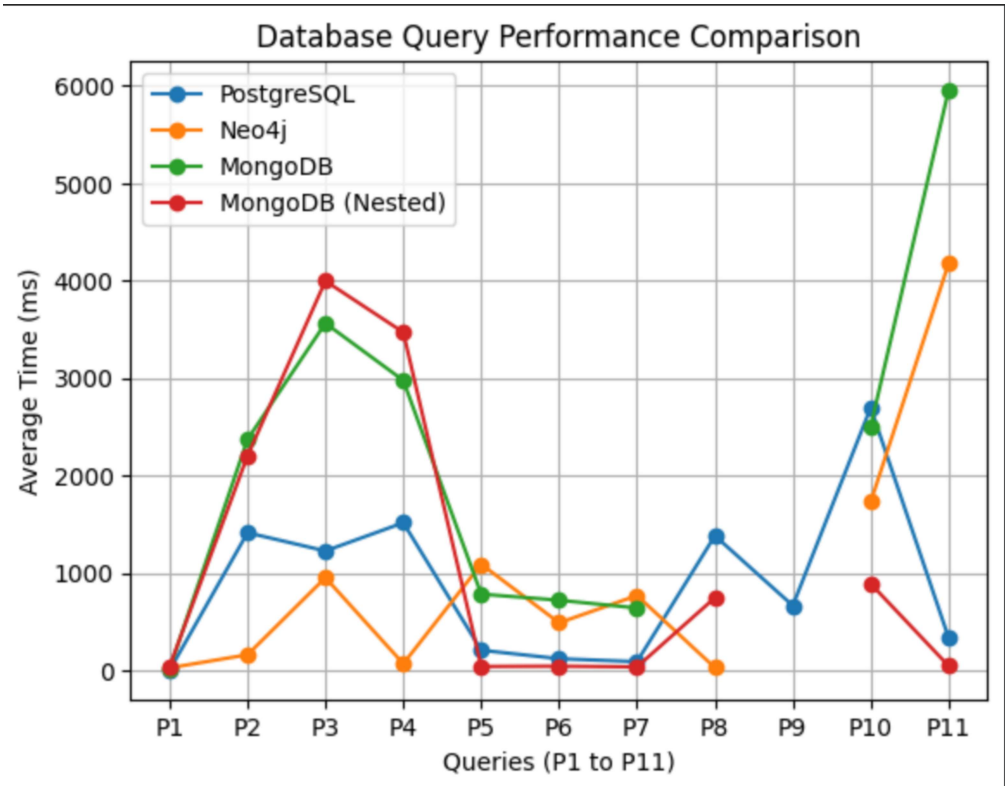


Figure 2: Database Query Performance Comparison

7 Inferences

List all tags used in more than 1,000 posts

We perform a sequential scan or index scan on a small Tags table with a WHERE Count $>$ 1000 filter. This query is very friendly for sql, hence postgresql performs the best.

Comments containing the word 'error'

Neo4j performs the best. It likely builds an inverted index which is a mapping from every word to the list of documents (nodes) containing that word.

Power Users with $\geq$ 1 Post and $\geq$ 1 Comment

Neo4j performs the best here. It can simply look at outgoing edges of each user and see if it finds CREATED_POST and WROTE_COMMENT. Postgresql has to do two scans and then a join. Mongodb naturally performs worse on join like operations. Nested MongoDB is even slower than flat here because comments are embedded inside posts inside users, so finding "users who have comments" requires unwinding nested arrays rather than scanning a flat Comments collection.

Active Users (Created Post or Left Comment)

The same reasonings apply as query 3 as the processing will be nearly identical.

Top 10 users by reputation in New York

Nested mongodb performs best here, because it is mostly a sequential algorithm on a single collection. Neo4j is particularly bad because relations are not used at all. It just has to traverse all the nodes to measure popularity.

Comments by "JohnDoe" Ordered by Date

Here the nesting property of Mongodb is fully utilised. It only has to look up John Doe's document and go iterate through the comments. Flat Mongodb has to scan the whole comments collection. PostgreSQL (115ms) requires a join between Comments and Users (to resolve DisplayName = 'JohnDoe') plus an ORDER BY on a date column Neo4j (310ms) traverses `User` $\rightarrow$ `WROTE_COMMENT` $\rightarrow$ `Comment` edges and sorts, which is reasonable but slower due to the per-relationship traversal cost and sorting.

Count distinct badge types for a user

Nested Mongodb wins as badges are nested inside users. Flat Mongodb performs poorly as it has to scan entire badges collection. Neo4j must traverse User $\rightarrow$ HAS_BADGE $\rightarrow$ Badge edges and count distinct names, carrying per-hop overhead.

Users who commented on their own posts > 5 times

Neo4j is considerably faster. It needs to find all paths of the form $User \rightarrow CREATED \rightarrow Post \leftarrow COMMENTED_ON \leftarrow User$ and count the occurrences. PostgreSQL needs a self-join. Flat MongoDB timed out entirely because without efficient joins, the solution may have a complexity of $O(N^2)$. Nested MongoDB works because comments are embedded within posts.

Accepted answers by elite users

Only PostgreSQL returned results. It is a heavy relational join hence handled well PostGreSQL. We had some trouble planning the query for neo4j.

User with post having the highest comment count

Nested MongoDB wins because the comment count is pre-computed or directly countable from embedded arrays. Neo4j can traverse $Post \rightarrow COMMENTED_ON$ edges and count. PostgreSQL and flat MongoDB are slower, both requiring a full aggregation across the Comments table grouped by `PostId`, followed by a join to Posts and Users, and then a sort to find the maximum.

User profile page (user $\rightarrow$ top 5 posts $\rightarrow$ 3 recent comments each)

Nested MongoDB performs significantly better. The access pattern is very suitable here and utilizes the nesting of posts and comments. Flat MongoDB is worst (5124ms) because it needs three separate queries or lookup stages to assemble the hierarchy from flat collections.

8 Conclusion

PostgreSQL delivered consistent, mid-range performance across nearly every query. Its internal query optimizer and hash join implementation make it the safest general-purpose choice.
Neo4j showed the most extreme variance between its best and worst results. For queries fundamentally about relationships between entities, it was much faster than the rest. However, it performed poorly on property-centric queries involving filtering, sorting, and limiting without any relationship traversal.
For Mongodb, schema design is very important. If it is given the same structure as in sql, then it performs the worst and basically as an inferior version of PostgreSQL.
After nesting, it performs the best for queries where data lives within a single document. Nested MongoDB

resolves everything in one document read with no joins whatsoever. But nesting actively hurts queries that cut across the embedding hierarchy: Q3 and Q4 (cross-entity intersection/union) were slower in nested than in flat MongoDB, because answering them requires unwinding deeply nested arrays rather than scanning a flat collection.

9 Challenges

- Selecting a sufficiently large dataset was difficult, but necessary to accurately measure and compare query execution times.
- While working with PostgreSQL, we faced foreign key constraint violations due to missing or inconsistent referenced data, which required additional preprocessing.
- Setting up MongoDB, Neo4j, and PostgreSQL was time-consuming, as each system has a different data model, configuration, and query language.

10 Future Scope

- Extend the study to include more database systems (e.g., distributed databases like Cassandra or cloud-based solutions) and evaluate performance under real-time and large-scale workloads.

11 Code Link

Project Files (Google Drive)